

Curso desarrollo de aplicaciones con tecnologías web
Módulo formativo 2: Programación web en el entorno servidor

Unidad formativa 1: Desarrollo de aplicaciones web en el entorno servidor

Lección 1: El proceso del desarrollo de software

Pruebas de software

Calidad del software

Herramientas de uso común para el desarrollo de software

Gestión de proyectos de desarrollo de software

Pruebas de software

1. Tipos de pruebas

Las pruebas se pueden clasificar en:

- **Pruebas estáticas:** Se realizan sin ejecutar el software, como la revisión de código o el análisis de documentación.
- **Pruebas dinámicas:** Implican la ejecución del software para comprobar su comportamiento.
- **Pruebas funcionales (caja negra):** Se centran en verificar que el sistema cumple con los requisitos funcionales, sin analizar su código interno.
- **Pruebas estructurales (caja blanca):** Evalúan el funcionamiento interno del código, verificando la ejecución de cada una de sus partes.

2. Pruebas funcionales (BBT - Black Box Testing)

Estas pruebas evalúan la funcionalidad de la aplicación sin conocer su código interno.

Ejemplo:

- **Casos de prueba:** Se prueba si el usuario puede registrarse correctamente con diferentes datos de entrada.
- **Herramientas:** Selenium, TestComplete.

3. Pruebas estructurales (WBT - White Box Testing)

En estas pruebas se analiza el código fuente, asegurando que todas las condiciones y bucles se ejecutan correctamente.

Ejemplo:

- Probar si una función de cálculo devuelve el resultado esperado para diferentes valores.
- **Herramientas:** PHPUnit, JUnit.

4. Comparativa y pautas de utilización

Característica	Pruebas funcionales (BBT)	Pruebas estructurales (WBT)
Conocimiento del código	No necesario	Requiere acceso al código
Enfoque	Comportamiento del sistema	Lógica interna y estructuras de control
Ejemplo	Validar login de usuario	Comprobar ejecución de una función

5. Diseño de pruebas

El diseño de pruebas consiste en definir los casos de prueba, datos de entrada, pasos a seguir y los resultados esperados.

6. Ámbitos de aplicación

Las pruebas pueden aplicarse en:

- **Pruebas de unidad** (cada módulo por separado).
- **Pruebas de integración** (combinación de módulos).
- **Pruebas de aceptación** (verifican si el software satisface los requisitos del cliente).

7. Pruebas de sistemas

Evalúan la aplicación completa en su entorno real.

Ejemplo: Verificar que un e-commerce procesa correctamente los pedidos.

8. Pruebas de componentes

Se enfocan en módulos específicos del software.

Ejemplo: Probar si la función de envío de correo funciona correctamente.

9. Automatización de pruebas. Herramientas

- **Selenium:** Para pruebas de interfaz web.
- **Jenkins:** Para integración continua y ejecución de pruebas automatizadas.
- **JUnit / PHPUnit:** Pruebas unitarias en Java y PHP.

10. Estándares sobre pruebas de software

- **ISO/IEC 29119:** Establece estándares para procesos de pruebas.
- **ISTQB:** Marco de certificación en pruebas de software.

Calidad del software

La calidad del software se refiere a su capacidad para cumplir con los requisitos funcionales y no funcionales de manera eficiente y confiable.

1. Principios de calidad del software

- **Fiabilidad:** El sistema no debe fallar en su uso normal.
- **Eficiencia:** Uso optimizado de recursos.
- **Mantenibilidad:** Facilidad para corregir errores y realizar mejoras.
- **Portabilidad:** Capacidad de ejecutarse en diferentes entornos.

2. Métricas y calidad del software

Las métricas permiten medir aspectos clave del software.

Ejemplo de métricas:

- Número de defectos por módulo.
- Cobertura de código en pruebas unitarias.

3. Concepto de métrica y su importancia

Las métricas permiten evaluar la calidad y tomar decisiones basadas en datos.

4. Principales métricas en las fases del ciclo de vida del software

- **En desarrollo:** Cantidad de errores detectados por fase.
- **En pruebas:** Porcentaje de cobertura de pruebas automatizadas.

5. Estándares para la descripción de los factores de calidad

- ISO/IEC 25010: Define características de calidad del software.

6. ISO-9126

Estándar anterior a ISO 25010 que clasificaba la calidad en funcionalidad, fiabilidad, usabilidad, eficiencia, mantenibilidad y portabilidad.

7. Otros estándares. Comparativa

- **ISO 9001:** Certificación de calidad general.
- **CMMI:** Modelo para evaluar la madurez de procesos de desarrollo.

Herramientas de uso común para el desarrollo de software

En el desarrollo de software, se utilizan diversas herramientas para facilitar la escritura, prueba, optimización y gestión del código. Estas herramientas ayudan a mejorar la productividad y la calidad del software.

1. Editores orientados a lenguajes de programación

Los editores de código son programas diseñados para escribir y gestionar código fuente. Algunos incluyen funciones avanzadas como resaltado de sintaxis, autocompletado y depuración.

✅ Ejemplos de editores populares:

- **VS Code (Visual Studio Code)** → Liger, extensible con plugins y soporte para múltiples lenguajes.
- **Sublime Text** → Rápido y personalizable, ideal para scripts rápidos.
- **Atom** → Editor con soporte de Git y desarrollado por GitHub (ahora en desuso en favor de VS Code).
- **Vim / Nano** → Editores en terminal para usuarios avanzados.

◆ Ejemplo de configuración en VS Code para PHP

Se puede agregar soporte a PHP con la extensión PHP Intelephense y configurar el formato automático en settings.json:

```
{  
    "editor.formatOnSave": true,  
    "php.validate.executablePath": "/usr/bin/php"  
}
```


2. Compiladores y enlazadores

Los compiladores traducen el código fuente a un lenguaje ejecutable por la máquina. Los enlazadores combinan varios archivos compilados en un solo programa ejecutable.

✓ Ejemplos de compiladores:

- **GCC (GNU Compiler Collection)** → Soporta C, C++, entre otros.
- **Javac** → Compilador de Java.
- **PHP** → No necesita compilación tradicional, pero se puede usar Opcache para optimizar el rendimiento.
- **TypeScript Compiler (TSC)** → Transpila TypeScript a JavaScript.

◆ Ejemplo de compilación en C++ con GCC:

bash:

```
g++ programa.cpp -o programa  
./programa
```

3. Generadores de programas

Automatizan la creación de código repetitivo o plantillas de proyectos.

✅ Ejemplos:

- **Yeoman** → Generación de estructuras de proyectos en JS.
- **Laravel Artisan** → Generador de código en Laravel (php artisan make:model).
- **Symfony Console** → Creación de comandos personalizados.

◆ Ejemplo en Laravel (PHP):

bash:

```
php artisan make:controller MiControlador
```

4. Depuradores

Los depuradores permiten analizar el código en tiempo de ejecución, identificar errores y seguir el flujo de ejecución.

✓ Ejemplos:

- **Xdebug** → Depuración en PHP con VS Code.
- **Chrome DevTools** → Depuración de JavaScript en navegadores.
- **GDB (GNU Debugger)** → Depuración en C/C++.

◆ Ejemplo de activación de Xdebug en PHP (archivo php.ini):

ini:

```
zend_extension=xdebug  
xdebug.mode=debug
```

5. Herramientas de prueba y validación de software

Permiten automatizar pruebas de funcionalidad y rendimiento.

✅ Ejemplos:

- **PHPUnit** → Pruebas unitarias en PHP.
- **Selenium** → Automatización de pruebas de interfaz web.
- **Jest / Mocha** → Pruebas en JavaScript.

◆ Ejemplo de prueba unitaria en PHPUnit:

php:

```
use PHPUnit\Framework\TestCase;
```

```
class CalculadoraTest extends TestCase {  
    public function testSuma() {  
        $this->assertEquals(4, 2+2);  
    }  
}
```

6. Optimizadores de código

Mejoran la eficiencia del código eliminando redundancias o precompilando scripts.

✓ Ejemplos:

- **PHP Opcache** → Acelera scripts PHP almacenando código compilado en memoria.
- **Google Closure Compiler** → Minimiza y optimiza código JS.
- **UglifyJS** → Compresión de JavaScript.

◆ Ejemplo de activación de Opcache en php.ini:
ini:

```
opcache.enable=1  
opcache.memory_consumption=128
```

7. Empaquetadores

Agrupan archivos en un solo paquete para distribución.

✓ Ejemplos:

- **Webpack** → Gestión de módulos en JS.
- **Composer** → Gestión de paquetes en PHP.
- **Parcel** → Alternativa a Webpack más sencilla.

◆ Ejemplo de uso de Composer en PHP:
bash:

```
composer require monolog/monolog
```

8. Generadores de documentación de software

Automatizan la creación de documentación a partir del código.

✅ Ejemplos:

- **PHPDoc** → Documentación automática de código PHP.
- **Swagger** → Documentación de APIs REST.
- **Doxygen** → Documentación para C/C++.

◆ Ejemplo de PHPDoc en un código PHP:

php:

```
/**  
 * Suma dos números.  
 *  
 * @param int $a Primer número.  
 * @param int $b Segundo número.  
 * @return int Resultado de la suma.  
 */  
function sumar($a, $b) {  
    return $a + $b;  
}
```

9. Gestores y repositorios de paquetes

Facilitan la instalación y gestión de dependencias de software.

✓ Ejemplos:

- **Composer (PHP)** → Gestión de paquetes en PHP.
- **NPM / Yarn** → Gestión de dependencias en JavaScript.
- **Pip** → Instalación de paquetes en Python.

◆ Ejemplo de instalación de una dependencia en NPM:

bash:

```
npm install axios
```

10. Distribución de software

Herramientas para empaquetar y distribuir aplicaciones.

✓ Ejemplos:

- **Docker** → Empaquetado en contenedores.
- **Electron** → Aplicaciones de escritorio con tecnologías web.
- **AppImage / Snap / Flatpak** → Distribución de apps en Linux.

11. Gestores de actualización de software

Automatizan la actualización de paquetes y dependencias.

✓ Ejemplos:

- **Composer Update** → Para PHP.
- **npm update** → Para Node.js.
- **Apt / Yum / Pacman** → Gestores de paquetes en Linux.

12. De control de versiones

Permiten gestionar cambios en el código fuente y colaborar en equipo.

✓ Ejemplos:

- **Git** → Control de versiones distribuido.
- **GitHub / GitLab / Bitbucket** → Hosting de repositorios.

◆ Ejemplo de flujo básico con Git:

bash:

```
git init
git add .
git commit -m "Primer commit"
git push origin main
```


13. Entornos integrados de desarrollo (IDE)

Incluyen editores, depuradores, compiladores y herramientas de integración.

 Ejemplos:

- **VS Code** → Multilenguaje y extensible.
- **PhpStorm** → Especializado en PHP.
- **Eclipse** → Soporte para múltiples lenguajes.
- **NetBeans** → Orientado a Java y PHP.

Gestión de proyectos de desarrollo de software

La **gestión de proyectos de desarrollo de software** es el proceso de planificar, organizar, ejecutar y controlar un proyecto de software para cumplir con los objetivos de tiempo, costo y calidad. Se utilizan diversas metodologías y herramientas para garantizar una entrega eficiente del producto.

1. Planificación de proyectos

La planificación es la fase inicial donde se definen los objetivos, requisitos, recursos y cronograma del proyecto.

◆ Pasos clave en la planificación:

- 1. Definir objetivos → ¿Qué se quiere lograr con el software?
- 2. Alcance del proyecto → ¿Qué funcionalidades incluirá?
- 3. Requisitos → ¿Cuáles son los requisitos funcionales y no funcionales?
- 4. Recursos → Asignación de equipo, presupuesto y herramientas.
- 5. Cronograma → Uso de metodologías como Diagrama de Gantt o SCRUM.
- 6. Riesgos → Identificar posibles obstáculos y planes de mitigación.

✅ Ejemplo de un cronograma en un proyecto ágil (SCRUM)

Sprint	Funcionalidad	Duración	Responsable
Sprint 1	Diseño UI + API básica	2 semanas	Equipo de diseño + backend
Sprint 2	Implementación de usuarios	2 semanas	Backend + frontend
Sprint 3	Integración y pruebas	2 semanas	QA + desarrollo

2. Control de proyectos

El control del proyecto implica el monitoreo y ajuste continuo del desarrollo para asegurar que se cumplan los plazos y objetivos.

✓ Técnicas de control:

- Seguimiento del cronograma con herramientas como Jira, Trello o ClickUp.
- Control de versiones con Git y GitHub Projects.
- Gestión de riesgos con reuniones de revisión periódicas (Sprint Review en Scrum).
- Medición de métricas como velocidad de desarrollo, bugs resueltos, etc.

◆ Ejemplo de uso de GitHub Projects

- To Do → Diseñar login
- In Progress → Implementación de API
- Done → Integración con la base de datos

bash:

```
git checkout -b feature/login
git commit -m "Añadir funcionalidad de login"
git push origin feature/login
```

3. Ejecución de proyectos

La ejecución consiste en llevar a cabo la planificación mediante el desarrollo y la entrega del software.

◆ Factores clave en la ejecución:

- Asignación eficiente de tareas entre los equipos de desarrollo, diseño y pruebas.
- Reuniones diarias (Daily Standups) en metodologías ágiles como Scrum.
- Uso de metodologías híbridas si se necesita una mezcla entre agilidad y planificación clásica (ejemplo: Scrum + Waterfall).

✅ Ejemplo de ejecución con Scrum

- 📌 Día 1: Sprint planning → Se definen las tareas para las siguientes 2 semanas.
- 📌 Día 2-10: Desarrollo y pruebas → Cada desarrollador trabaja en su tarea asignada.
- 📌 Día 11: Sprint review → Se presenta el avance y se recibe feedback.
- 📌 Día 12: Sprint retrospective → Se analizan mejoras en el proceso.

4. Herramientas de uso común para la gestión de proyectos

1 Herramientas de gestión de tareas

- Trello → Tableros visuales para tareas (Kanban).
- Jira → Gestión ágil de proyectos para equipos grandes.
- Asana → Planificación de tareas con flujos de trabajo.

◆ Ejemplo de un tablero Kanban en Trello:

[🔴 Backlog] → [🟡 En progreso] → [🟢 Completado]

2 Control de versiones y colaboración

- GitHub / GitLab → Control de versiones con Git.
- Bitbucket → Alternativa a GitHub con integración en Jira.
- Azure DevOps → Para equipos que trabajan con Microsoft.

◆ Ejemplo de flujo de trabajo con Git:

bash:

```
git branch feature/registro
git checkout feature/registro
# Desarrollo de código...
git commit -m "Añadida funcionalidad de registro"
git push origin feature/registro
```

Documentación y comunicación

- Confluence → Documentación colaborativa.
- Notion → Gestión de notas y tareas.
- Slack / Microsoft Teams → Comunicación entre equipos.

Ejemplo de documentación en Confluence

 Título: Guía de instalación del backend

 Secciones:

- Requisitos
- Instalación
- Ejemplos de uso

4 Integración y despliegue continuo (CI/CD)

- Jenkins → Automatiza el despliegue y pruebas.
- GitHub Actions → Ejecución automática de pruebas en GitHub.
- Docker → Contenedores para empaquetado del software.

◆ Ejemplo de archivo `.github/workflows/main.yml` para CI/CD

yaml:

```
name: CI/CD Pipeline
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - run: npm install
      - run: npm test
```

5. Conclusión

- 📌 La gestión de proyectos de software es clave para garantizar el éxito del desarrollo.
- 📌 SCRUM y Kanban son metodologías ágiles que mejoran la organización.
- 📌 GitHub, Trello, Jira y otras herramientas facilitan la planificación, control y ejecución.
- 📌 La automatización (CI/CD, GitHub Actions) permite entregas más rápidas y confiables.